Interro blanche — Corrigé

Parties 02 (Codage des nombres) et 03 (En mémoire — IEEE 754)

Partie A — Le codage des nombres (/32)

Question 1 — Opérations en binaire (/4 — 1 pt par opération)

```
a)    1 1 1        <- retenues
        1 0 1 1 0
    +     1 1 0 1
    -----------
      1 0 0 0 1 1   (22 + 13 = 35)

b)  1 1 1 1 1       <- retenues
      1 1 0 1 1
    + 1 0 1 1 1
    -----------
    1 1 0 0 1 0   (27 + 23 = 50)

c)    1 1 0 0 1
    - 0 1 0 1 1
    -----------
      0 1 1 1 0     (25 − 11 = 14)

d)    1 0 0 0 0
    - 0 0 1 1 1
    -----------
      0 1 0 0 1   (16 − 7 = 9)
```

1. pos1 : 0 − 1 → emprunt en cascade à travers pos2 (0) jusqu'à pos3 ; pos3 (devenu 0) : 0 − 1 → nouvel emprunt sur pos4. Résultat **1110**.
2. cascade d'emprunts à travers les 4 zéros (comme 1000 − 348). Résultat **1001**.

Question 2 — Du binaire vers la base 10 (/4)

Binaire	**Somme des poids**	**Décimal**
110101	32 + 16 + 4 + 1	**53**
10000001	128 + 1	**129**
11100111	128 + 64 + 32 + 4 + 2 + 1	**231**
1011,101	8 + 2 + 1 + 0,5 + 0,125	**11,625**

Question 3 — De la base 10 vers le binaire (/4)

Décimal	**Binaire**	**Remarque**
45	**101101**	restes : 1,0,1,1,0,1 (lus de bas en haut)
200	**11001000**	128 + 64 + 8
127	**1111111**	$2^7 - 1$: que des 1
1000	**1111101000**	512 + 256 + 128 + 64 + 32 + 8

Question 4 — Complément à 1, sur 5 bits (/4)

a) $(11)_{10}$ = **01011** ; $(-11)_{10}$: 01011 inversé → **10100**.

b) 10101 : bit de gauche = 1 → négatif ; inversion → 01010 = 10 → **−10**.

```
  0 1 0 0 1   (+9)
+ 1 1 0 1 1   (−4 : 00100 inversé)
-----------
1 0 0 1 0 0   <- retenue sortante = 1
report circulaire : 00100 + 1 = 00101  →  (+5)
```

La retenue qui sort à gauche doit être réinjectée (ajoutée au bit de droite), sinon le résultat serait faux (4 au lieu de 5).

d) Il existe **deux zéros** : +0 = 00000 et −0 = 11111. Ex. : 6 + (−6) = 00110 + 11001 = 11111 = −0. (L'obligation du report circulaire est un 2e défaut accepté.)

Question 5 — Complément à 2, sur 8 bits (/5)

Nombre		**valeur**	
−37	00100101	11011010	**11011011**
−90	01011010	10100101	**10100110**
64	positif → écriture directe		**01000000**
Mot	**Signe**	**Calcul**	**Valeur**
1110 0100	négatif	inversion 0001 1011, +1 → 0001 1100 = 28 (ou 228 − 256)	**−28**
0101 1010	positif	64 + 16 + 8 + 2	**90**
1000 0001	négatif	inversion 0111 1110, +1 → 0111 1111 = 127 (ou 129 − 256)	**−127**

Question 6 — Nombres réels en binaire (/4)

a) **9,625** : 9 = 1001 ; 0,625 × 2 = 1,25 → 1 ; 0,25 × 2 = 0,5 → 0 ; 0,5 × 2 = 1 → 1 → **$(1001{,}101)_2$** (exact).

0,3 : 0,6→0 ; 1,2→1 ; 0,4→0 ; 0,8→0 ; 1,6→1 ; 1,2→1 ; 0,4→0 ; 0,8→0 ; 1,6→1 ; 1,2→1 ; 0,4→0 ; 0,8→0 → **$(0{,}010011001100\ldots)_2$**. Le motif 1001 se répète : le développement est **infini**, 0,3 ne peut pas être codé exactement (erreur d'arrondi).

b) $(101{,}0011)_2$ = 4 + 1 + 0,125 + 0,0625 = **5,1875** ; $(0{,}0101)_2$ = 0,25 + 0,0625 = **0,3125**.

Question 7 — L'hexadécimal (/4)

Hexa	**Binaire (16 bits)**	**Décimal**
B3E	0000 1011 0011 1110	11×256 + 3×16 + 14 = **2878**
F00D	1111 0000 0000 1101	15×4096 + 13 = **61453**

b) 1101 0110 0011 → D, 6, 3 → **$(D63)_{16}$** ; 101101 → on complète à gauche : 0010 1101 → **$(2D)_{16}$**.

Question 8 — Réflexion (/3)

a) Sur 16 bits signés : de $\mathbf{-2^{15} = -32\,768}$ à $\mathbf{2^{15} - 1 = +32\,767}$. Sur un octet signé (−128 à +127) : −129 **NON** (< −128) ; 100 **OUI** (01100100).

b) $(C4A2)_{16}$ = 1100 0100 1010 0010.

- Non signé : 12×4096 + 4×256 + 10×16 + 2 = **50 338**.
- Signé : bit de gauche = 1 → négatif ; 50 338 − 65 536 = **−15 198** (ou inversion 0011 1011 0101 1101 + 1 = 15 198).

c) B = 66 → 01000010 ; i = 105 → 01101001 ; t = 116 → 01110100 → **01000010 01101001 01110100** (3 octets).

Partie B — En mémoire : la norme IEEE 754 (/18)

Question 9 — Coder en IEEE 754 (/8 — 2 pts par nombre)

a) −21,375 1) S = 1 ; 2) 21 = 10101 ; 3) 0,375 = 0,011 (fini) ; 4) 10101,011 = 1,0101011 × 2^4 ; 5) E = 4 + 127 = 131 = 10000011 ; 6) M = 0101011 complétée. → **1 10000011 01010110000000000000000**

b) 0,6875 1) S = 0 ; 2) partie entière 0 ; 3) 0,6875 = 0,5 + 0,125 + 0,0625 = 0,1011 ; 4) 0,1011 = 1,011 × 2^{-1} ; 5) E = −1 + 127 = 126 = 01111110 ; 6) M = 011… → **0 01111110 01100000000000000000000**

c) 300,25 1) S = 0 ; 2) 300 = 256 + 32 + 8 + 4 = 100101100 ; 3) 0,25 = 0,01 ; 4) 100101100,01 = 1,0010110001 × 2^8 ; 5) E = 8 + 127 = 135 = 10000111 ; 6) M = 0010110001… → **0 10000111 00101100010000000000000**

d) −0,1 1) S = 1 ; 2) partie entière 0 ; 3) 0,1 = 0,000110011001100… (infini) ; 4) premier 1 au rang 2^{-4} → 1,10011001100… × 2^{-4} ; 5) E = −4 + 127 = 123 = 01111011 ; 6) M = 1001 1001 1001 1001 1001 100|11… → arrondi au plus proche : 10011001100110011001101. → **1 01111011 10011001100110011001101** (valeur stockée ≈ −0,100000001490…, non exacte)

Question 10 — Décoder un mot IEEE 754 (/6 — 2 pts par mot)

a) S = 0 (positif) ; E = 10000011 = 131 → e = 4 ; 1,M = 1,01101 ; × 2^4 → 10110,1 = 16 + 4 + 2 + 0,5 → **22,5**

b) S = 1 (négatif) ; E = 01111101 = 125 → e = −2 ; 1,M = 1,1 ; × 2^{-2} → 0,011 = 0,25 + 0,125 → **−0,375**

c) S = 0 (positif) ; E = 10000101 = 133 → e = 6 ; 1,M = 1,0010101 ; × 2^6 → 1001010,1 = 64 + 8 + 2 + 0,5 → **74,5**

Question 11 — Virgule fixe ou flottante ? (/2)

Nombre	**IEEE 754 (stocké / erreur)**	**Q16.16 (stocké / erreur)**	**Plus précis**
0,2	0,200000003 / ≈ $3 \cdot 10^{-9}$	0,199996948 / ≈ $3 \cdot 10^{-6}$	**IEEE 754** (~1000× mieux)
5000,3	5000,2998047 / ≈ $2 \cdot 10^{-4}$	5000,3000031 / ≈ $3 \cdot 10^{-6}$	**Virgule fixe**
80000,5	80000,5 / 0 (exact)	**hors plage** (> 65 535)	**IEEE 754**

Justification attendue : en IEEE 754 la précision est **relative** (24 bits significatifs, le pas grandit avec la valeur) : excellente pour les petits nombres, moins bonne pour la partie décimale des grands nombres. En virgule fixe le pas est **constant** ($2^{-16} \approx 0{,}000015$) mais la plage est limitée à 65 535.

Question 12 — Réflexion (/2)

a) Le biais permet de coder des exposants positifs **et** négatifs avec un simple nombre non signé (E = e + 127), ce qui rend la comparaison de deux flottants aussi simple que celle de deux entiers et évite les problèmes de signe. Pour un nombre entre 0 et 1, e < 0 donc **E < 127** (ex. 0,5 → E = 126).

b) 0,1 et 0,2 ont un **développement binaire infini** (motif 0011 répété) : ils sont tronqués/arrondis dans la mantisse de 23 bits. Les petites erreurs d'arrondi s'additionnent, et le résultat est très proche de 0,3 mais pas égal (0,30000000000000004 en double précision).